

Kommunikationsprotokoll för IoT-passagesystem

Elevprojekt

Version 1.0

Innehåll

1 Syfte	3
2 Systemarkitektur	3
2.1 Variant A: Lokal Python-backend	3
2.2 Variant B: ESP32 som gateway	3
3 Ansvarsfördelning	3
3.1 Arduino	3
3.2 Backend	4
4 UART-inställningar	4
5 Ramformat	4
5.1 Fältbeskrivning	4
6 Payload-format	5
6.1 Fält i payload	5
6.2 Exempel	5
7 Sessions-ID	5
7.1 Regler för sessions-ID	5
8 Kommandon	6
9 NACK-koder	6
10 Checksumma	6
10.1 Pseudokod	7
10.2 Exempel	7
11 Kommunikationsflöde	7
11.1 Godkänd passage	7
11.2 Neka passage	7
11.3 Okänt kort	7
11.3.1 Strategi A: Direkt neka okänt kort	7
11.3.2 Strategi B: Begär alltid PIN	8
12 Tillståndsmaskin för Arduino	8
13 Tillståndsmaskin för backend	9
14 Exempel på komplett UART-frame	9

15 RFID och UID	9
16 Säkerhetsbegränsningar	10
17 Möjliga förbättringar	10
18 Sammanfattning	10

1 Syfte

Detta dokument beskriver ett kommunikationsprotokoll för ett IoT-baserat passagesystem.

Systemet består av en Arduino Uno med RFID-läsare och pinkodsinmatning. Arduino-enheten kommunicerar över UART med en backend. Backend kan antingen vara ett lokalt Pythonprogram eller en ESP32 som skickar vidare information till en webserver.

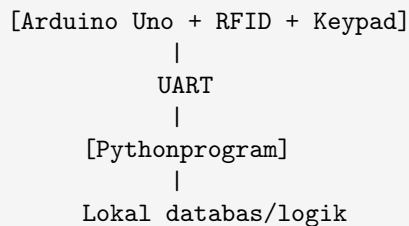
Protokollet används för att:

- skicka UID från RFID-taggar,
- begära pinkod,
- validera UID och PIN i backend,
- returnera beslut om åtkomst,
- hantera fel och ogiltiga meddelanden.

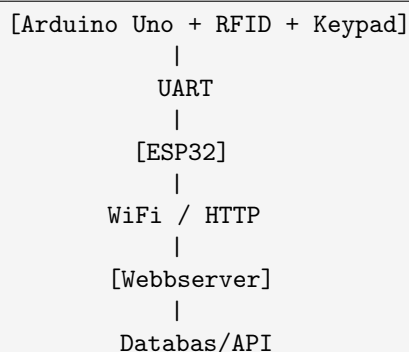
2 Systemarkitektur

Systemet kan användas i två huvudsakliga varianter.

2.1 Variant A: Lokalt Python-backend



2.2 Variant B: ESP32 som gateway



Arduino-enheten ska inte innehålla databasen. Den ska endast läsa RFID, läsa PIN, skicka meddelanden och reagera på svar från backend.

3 Ansvarsfördelning

3.1 Arduino

Arduino ansvarar för:

- att läsa UID från RFID-taggen,
- att läsa pinkod från keypad,
- att skicka UID och PIN över UART,
- att ta emot svar från backend,
- att styra indikatorer som LED, summer och relä/lås.

3.2 Backend

Backend ansvarar för:

- att lagra giltiga UID:n,
- att lagra eller kontrollera PIN-koder,
- att skapa sessions-ID,
- att validera UID och PIN,
- att hantera felaktiga försök,
- att skicka beslut tillbaka till Arduino.

4 UART-inställningar

Kommunikationen sker via UART med följande inställningar:

Parameter	Värde
Baudrate	9600
Databitar	8
Paritet	Ingen
Stoppbitar	1
Flödeskontroll	Ingen

Tabell 1: UART-inställningar

5 Ramformat

Alla meddelanden skickas som binära ramar över UART.

STX	LEN	PAYLOAD	CHK	ETX
-----	-----	---------	-----	-----

5.1 Fältbeskrivning

Fält	Storlek	Beskrivning
STX	1 byte	Startbyte. Har värdet 0x02.
LEN	1 byte	Antal bytes i payload.
PAYLOAD	LEN bytes	ASCII-kodad nyttolast enligt formatet V1 SID COMMAND DATA.

CHK	1 byte	Checksumma beräknad som XOR av alla bytes i payload.
ETX	1 byte	Slutbyte. Har värdet 0x03.

6 Payload-format

Alla payloads ska följa samma struktur:

V1 SID COMMAND DATA

6.1 Fält i payload

Fält	Beskrivning
V1	Protokollversion.
SID	Sessions-ID. Värdet 0 betyder att ingen session ännu finns.
COMMAND	Kommando, exempelvis UID, PIN, OK eller ERR.
DATA	Data kopplad till kommandot. Kan vara tomt.

6.2 Exempel

```
V1|0|UID|04A1B2C3
V1|42|REQ_PIN|
V1|42|PIN|1234
V1|42|OK|
V1|42|ERR|
V1|42|LOCKED|
V1|0|PING|
V1|0|PONG|
```

7 Sessions-ID

Sessions-ID skapas av backend när Arduino skickar ett UID.

Första meddelandet använder alltid sessions-ID 0:

V1 0 UID 04A1B2C3

Backend skapar sedan en session och svarar med ett nytt sessions-ID:

V1 42 REQ_PIN

Efter detta ska Arduino använda samma sessions-ID när PIN skickas:

V1 42 PIN 1234

7.1 Regler för sessions-ID

- SID = 0 betyder att ingen session finns.
- Backend skapar sessions-ID.
- Arduino får inte hitta på egna sessions-ID.

- Samma sessions-ID ska användas för UID- och PIN-flödet.
- En session bör ha timeout.
- En session bör avslutas efter OK, ERR, LOCKED eller TIMEOUT.

8 Kommandon

Kommando	Riktning	Beskrivning
UID	Arduino → Backend	Skickar RFID-taggens UID.
REQ_PIN	Backend → Arduino	Begär pinkod från användaren.
PIN	Arduino → Backend	Skickar användarens pinkod.
OK	Backend → Arduino	Åtkomst godkänd.
ERR	Backend → Arduino	Åtkomst nekad.
LOCKED	Backend → Arduino	Kortet eller sessionen är låst efter för många felaktiga försök.
TIMEOUT	Backend → Arduino	Sessionen har gått ut.
RESET	Båda riktningar	Återställ aktuell session eller protokolltillstånd.
PING	Båda riktningar	Kontrollera att andra sidan svarar.
PONG	Båda riktningar	Svar på PING.
NACK	Båda riktningar	Felaktigt eller ogiltigt meddelande. Orsak anges i DATA.

9 NACK-koder

När ett meddelande inte kan behandlas kan mottagaren svara med NACK.

V1|42|NACK|BAD_CHECKSUM

Kod	Betydelse
BAD_CHECKSUM	Checksumman stämmer inte.
BAD_LENGTH	Längdfältet är ogiltigt.
UNKNOWN_COMMAND	Kommandot är okänt.
INVALID_STATE	Kommandot skickades i fel tillstånd.
TIMEOUT	Sessionen har gått ut.
BAD_FORMAT	Payload följer inte formatet V1 SID COMMAND DATA.

10 Checksumma

Checksumman beräknas som XOR av samtliga bytes i payload.

10.1 Pseudokod

```
uint8_t checksum = 0;

for each byte in payload:
    checksum = checksum ^ byte;
```

10.2 Exempel

För payload:

```
V1|0|UID|04A1B2C3
```

beräknas checksumman genom att XOR:a alla ASCII-tecken i payload.

11 Kommunikationsflöde

11.1 Godkänd passage

```
Arduino -> Backend:
V1|0|UID|04A1B2C3

Backend -> Arduino:
V1|42|REQ_PIN|

Arduino -> Backend:
V1|42|PIN|1234

Backend -> Arduino:
V1|42|OK|
```

11.2 Neka passage

```
Arduino -> Backend:
V1|0|UID|04A1B2C3

Backend -> Arduino:
V1|42|REQ_PIN|

Arduino -> Backend:
V1|42|PIN|9999

Backend -> Arduino:
V1|42|ERR|
```

11.3 Okänt kort

Det finns två möjliga strategier.

11.3.1 Strategi A: Direkt neka okänt kort

```
Arduino -> Backend:  
V1|0|UID|DEADBEEF  
  
Backend -> Arduino:  
V1|0|ERR|
```

11.3.2 Strategi B: Begär alltid PIN

```
Arduino -> Backend:  
V1|0|UID|DEADBEEF  
  
Backend -> Arduino:  
V1|43|REQ_PIN|  
  
Arduino -> Backend:  
V1|43|PIN|1234  
  
Backend -> Arduino:  
V1|43|ERR|
```

Strategi B avslöjar mindre information eftersom systemet inte direkt visar om UID finns i databasen.

12 Tillståndsmaskin för Arduino

```
IDLE  
|  
| Kort blippas  
v  
SEND_UID  
|  
v  
WAIT_BACKEND_RESPONSE  
|  
+--> REQ_PIN ----> WAIT_USER_PIN  
|  
+--> ERR -----> ACCESS_DENIED  
|  
+--> NACK -----> ERROR  
|  
v  
SEND_PIN  
|  
v  
WAIT_ACCESS_RESULT  
|  
+--> OK -----> ACCESS_GRANTED  
|  
+--> ERR -----> ACCESS_DENIED  
|  
+--> LOCKED -----> ACCESS_LOCKED  
|  
+--> TIMEOUT ----> SESSION_TIMEOUT
```


13 Tillståndsmaskin för backend

```
IDLE
|
|  UID mottaget
v
CREATE_SESSION
|
v
SEND_REQ_PIN
|
v
WAIT_PIN
|
+--> Korrekt UID + PIN ----> SEND_OK
|
+--> Fel UID eller PIN ----> SEND_ERR
|
+--> Fr mnga frsk -----> SEND_LOCKED
|
+--> Timeout -----> SEND_TIMEOUT
```

14 Exempel på komplett UART-frame

Payload:

```
V1|0|UID|04A1B2C3
```

Payload-längd:

```
18 bytes = 0x12
```

Frame:

```
STX LEN PAYLOAD CHK ETX
```

Representerat som bytes:

```
0x02 0x12 0x56 0x31 0x7C 0x30 0x7C 0x55 0x49 0x44 0x7C
0x30 0x34 0x41 0x31 0x42 0x32 0x43 0x33 CHK 0x03
```

Där CHK är XOR av alla bytes i:

```
V1|0|UID|04A1B2C3
```

15 RFID och UID

RFID-taggen identifieras med sitt UID. UID skickas som hexadecimal ASCII-sträng.

Exempel:

```
04A1B2C3
AABBCCDD
11223344
```

UID ska behandlas som en identifierare, inte som ett lösenord. Åtkomst ska därför inte godkännas enbart baserat på UID.

16 Säkerhetsbegränsningar

Detta protokoll är avsett för utbildning och demonstration.

Begränsningar:

- UID skickas i klartext.
- PIN skickas i klartext.
- UART skyddar inte mot fysisk avlyssning.
- Ingen kryptering används.
- Ingen challenge-response används.
- UID kan i vissa RFID-system kopieras eller simuleras.

Systemet bör därför inte användas i skarp drift utan ytterligare säkerhetsmekanismer.

17 Möjliga förbättringar

Följande förbättringar kan införas:

- krypterad kommunikation mellan ESP32 och webbserver,
- hashning av PIN i backend,
- timeout för sessioner,
- begränsat antal PIN-försök,
- loggning av passageförsök,
- meddelande-ID för varje frame,
- ACK/NACK för varje mottaget meddelande,
- challenge-response i stället för att skicka PIN i klartext,
- admin-kommandon för att lägga till och ta bort kort.

18 Sammanfattning

Protokollet använder ett fast ramformat:

STX LEN PAYLOAD CHK ETX

Payload har alltid strukturen:

V1 SID COMMAND DATA

Det gör protokollet:

- enkelt att parse,
- konsekvent,
- lämpligt för UART,
- användbart både med Python-backend och ESP32-gateway,
- tydligt för undervisning i embedded, IoT och kommunikationsprotokoll.